Intro to Data Science - HW 6

Copyright Jeffrey Stanton 2025, Jeffrey Saltz, Christopher Dunham, and Jasmina Tacheva

```
# Enter your name here: Mars Schrag
```

USE OF ARTIFICIAL INTELLIGENCE: While you are allowed to use generative AI as a learning tool for this assignment (e.g., to help with brainstorming, to see how to code a certain concept, to help identify flaws in reasoning, or to spot confusing or underdeveloped code segments), it must not be used to input homework questions, problems, or instructions directly, and you may not copy and paste any AI-generated content into your work. By submitting this assignment, you confirm that all the work is your own.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this homework by myself, with help from the book and the professor.
```

Data visualization is important because many people can understand data better when it is presented graphically. As a data scientist, you will have to present complex data to decision makers in a way they can understand. Based on lecture and your experience with Excel and other tools, you know there are numerous data visualization types (e.g., line charts). How many of them can you name?

The most powerful tool for data visualization in R is called ggplot. Written by computer/data scientist Hadley Wickham, this graphics grammar tool builds visualizations in layers. This method provides immense flexibility, but takes a bit of practice to master.

Step 1: Make a copy of the data

A. Read the **Syracuse Code Violations** data from this URL: "https://data-science-intro.s3.us-east-2.amazonaws.com/Code_Violations.csv" into a dataframe called **violations_df**.

```
violations_df <- read.csv("https://data-science-intro.s3.us-east-2.amazonaws.com/Code_Violations.csv")
```

B. Your dataframe, **violations_df**, contains a list of code violations recorded repeatedly over time (2017-2024). Identify the variable that captures the date a violation has occurred and inspect it. What is the level of detail captured in this variable (weekly, monthly, etc.)? Answer in a comment.

```
head(violations_df$violation_date)
```

```
## [1] "2023/11/28 12:33:12.093+00" "2023/04/21 15:16:21.407+00"
## [3] "2023/04/21 15:14:36.107+00" "2023/04/21 15:10:38.397+00"
## [5] "2018/12/31 10:05:13.410+00" "2018/12/31 10:05:13.410+00"
```

```
# it has year, month, date, and extra numbers
```

C. What is the data type of this variable? Explain in a comment and show the command you used to find out.

```r
class(violations_df$violation_date)
```

```
## [1] "character"
```

```r
# character, not listed as a date
```

D. To properly display the time values in our plots as dates, we must convert the time variable to date format using the **as.Date()** function. Run the code below, comment on how it works, and double-check the variable's data type to ensure it is no longer coded as text.

```r
violations_df$violation_date <- as.Date(violations_df$violation_date)
# Comment: it changes the data from character to a date
```

```r
class(violations_df$violation_date)
```

```
## [1] "Date"
```

Step 2: Dealing with missing data and subsetting

A. When you first start working with a dataset, it is always a good idea to look for missing values. Check the variables **number_violations** and **zip_median_income** for missing values using the **is.na()** function. Are there any?

```r
sum(is.na(violations_df$number_violations))
```

```
## [1] 0
```

```r
sum(is.na(violations_df$zip_median_income))
```

```
## [1] 22
```

```r
# there are missing values in zip median income, not in number violations
```

B. **imputeTS** is a R package that is specifically designed to 'repair' missing values in time series data. We'll use this instead of the simpler **mean substitution** method because it's more accurate. If there are missing values in the two variables, use the na_interpolation() function from the imputeTS package to interpolate the missing values. Make sure to first sort the dataframe by violation_date and then by complaint_zip before applying the function.

```r
# install.packages('imputeTS')
library(imputeTS)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
violations_df <- violations_df[order(violations_df$violation_date, violations_df$complaint_zip),
    ]

violations_df$number_violations <- na_interpolation(violations_df$number_violations)
violations_df$zip_median_income <- na_interpolation(violations_df$zip_median_income)
```

C. To ensure that there is no more missing data, run the code from Step 2A again:

```
sum(is.na(violations_df$number_violations))
```

```
## [1] 0
```

```
sum(is.na(violations_df$zip_median_income))
```

```
## [1] 0
```

D. Create a new dataframe containing only the data for the "Northside" neighborhood using subsetting.

Hint: To avoid including rows with missing values, add this condition to your subsetting code: & !is.na(violations_df$Neighborhood).For example, filtering for the "Southside" neighborhood would look like this : [violations_df$Neighborhood == "Southside" & !is.na(violations_df$Neighborhood),]

```
northside_df <- violations_df[violations_df$Neighborhood == "Northside" & !is.na(violations_df$Neighborh
    ]
```

E. Using the same logic, create a separate dataframe for the following neighborhood: Westcott

```
westcott_df <- violations_df[violations_df$Neighborhood == "Westcott" & !is.na(violations_df$Neighborho
    ]
```

F. Using the same logic, create a separate dataframe for the following neighborhood: Downtown

```
downtown_df <- violations_df[violations_df$Neighborhood == "Downtown" & !is.na(violations_df$Neighborho
    ]
```

G. Finally, using the same logic, create a separate dataframe for the University area which combines two neighborhoods: University Neighborhood and University Hill.

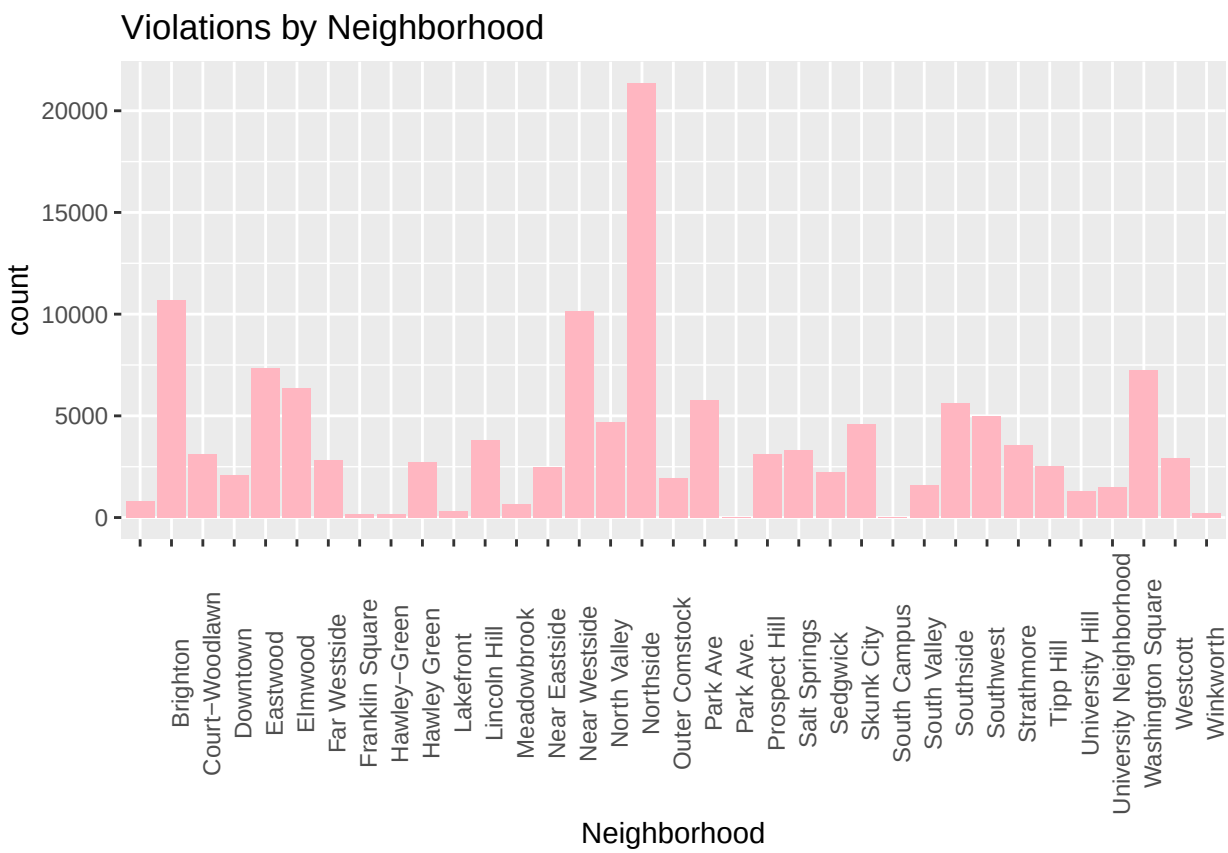
```
university_df <- violations_df[violations_df$Neighborhood %in% c("University Neighborhood",
    "University Hill") & !is.na(violations_df$Neighborhood), ]
```

Step 3: Use ggplot to explore the distribution of the data

Don t forget to install and library the ggplot2 package. Then: A. Create a bar chart for **Neighborhood** in the **violations df** dataframe. Make sure to add a title. In a comment, briefly interpret the histogram.

```
library(ggplot2)

ggplot(violations_df, aes(x = Neighborhood)) + geom_bar(fill = "lightpink") + labs(title = "Violations
    theme(axis.text.x = element_text(angle = 90))
```




```
# this histogram counts violations per neighborhood
```

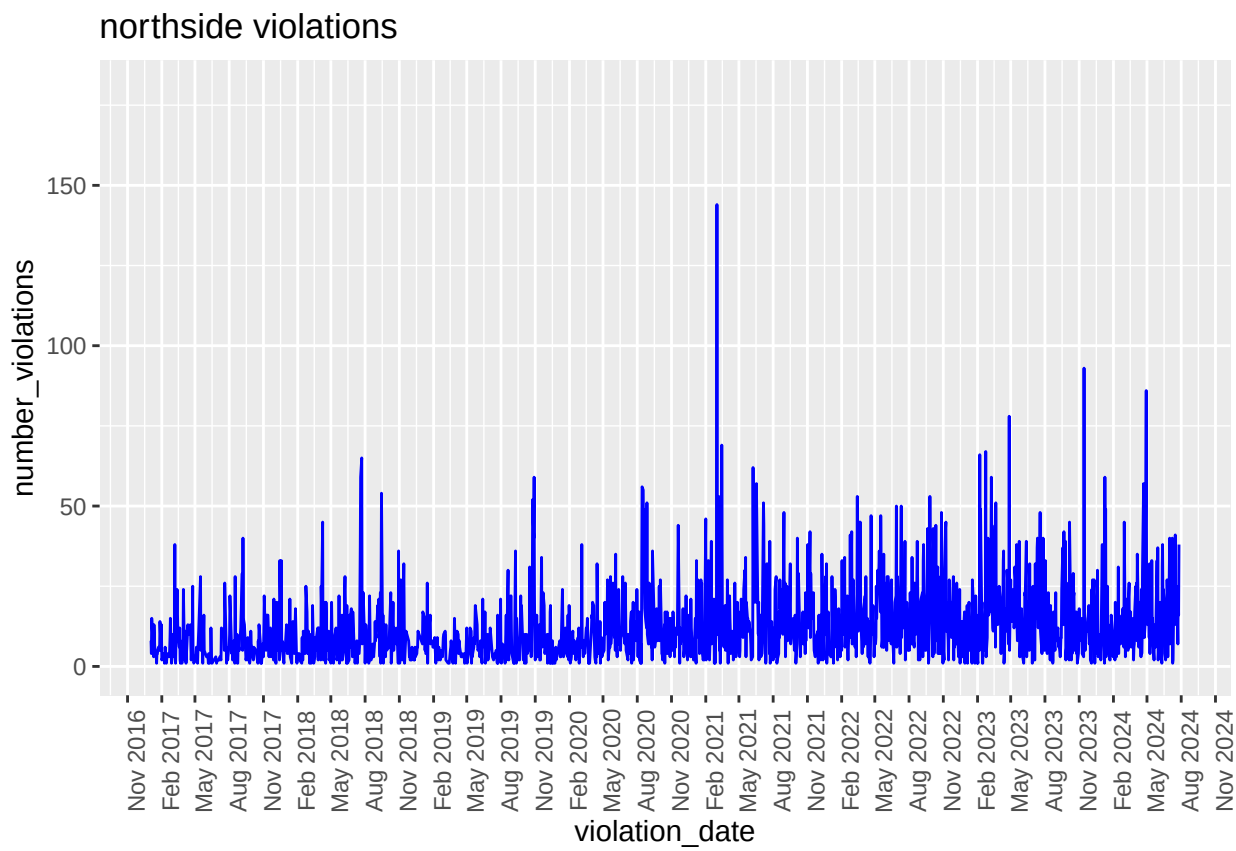
Step 4: Explore how the data changes over time

A. The data were collected over nearly 7 years. A line chart can be used to track changes over time. First, however, we need to aggregate the number of violations by date using the aggregate() function. Here is an example of how it works, applied to the overall data stored in violations_df. Use the same logic to obtain date aggregates for the four neighborhood dataframes:

```
violations_by_date <- aggregate(number_violations ~ violation_date, data = violations_df,
    sum)
date_northside <- aggregate(number_violations ~ violation_date, data = northside_df,
    sum)
date_university <- aggregate(number_violations ~ violation_date, data = university_df,
    sum)
date_westcott <- aggregate(number_violations ~ violation_date, data = westcott_df,
    sum)
date_downtown <- aggregate(number_violations ~ violation_date, data = downtown_df,
    sum)
```

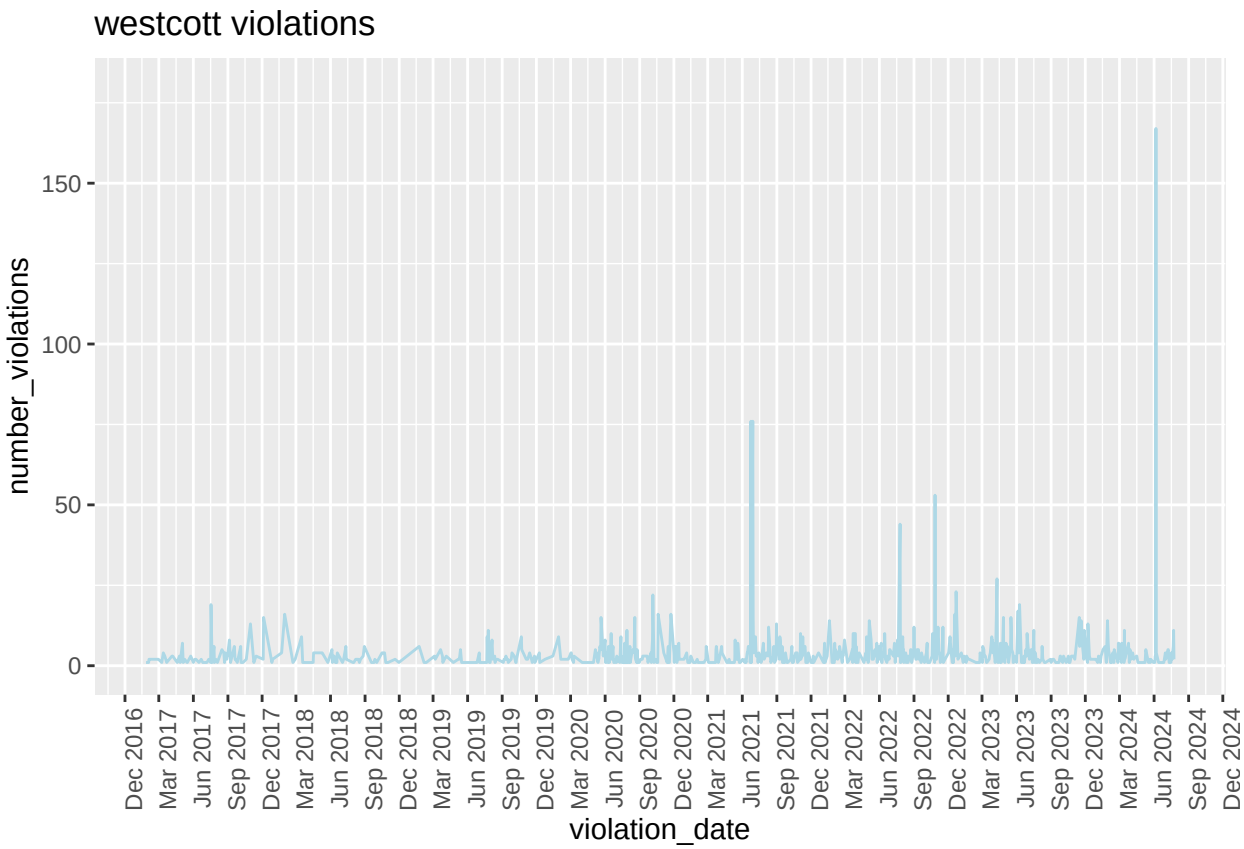
Once you are done with the previous step, we can start creating line charts to visualize the pattern of violations in each neighborhood over time. Let's focus on the **Northside df** subset first: Set the violation date variable on the X-axis and number_violations on the Y-axis of a **line chart**.

```r
ggplot(date_northside, aes(x = violation_date, y = number_violations)) + geom_line(color = "blue") +
    labs(title = "northside violations") + ylim(0, 180) + scale_x_date(date_breaks = "3 months",
    date_labels = "%b %Y") + theme(axis.text.x = element_text(angle = 90))
```

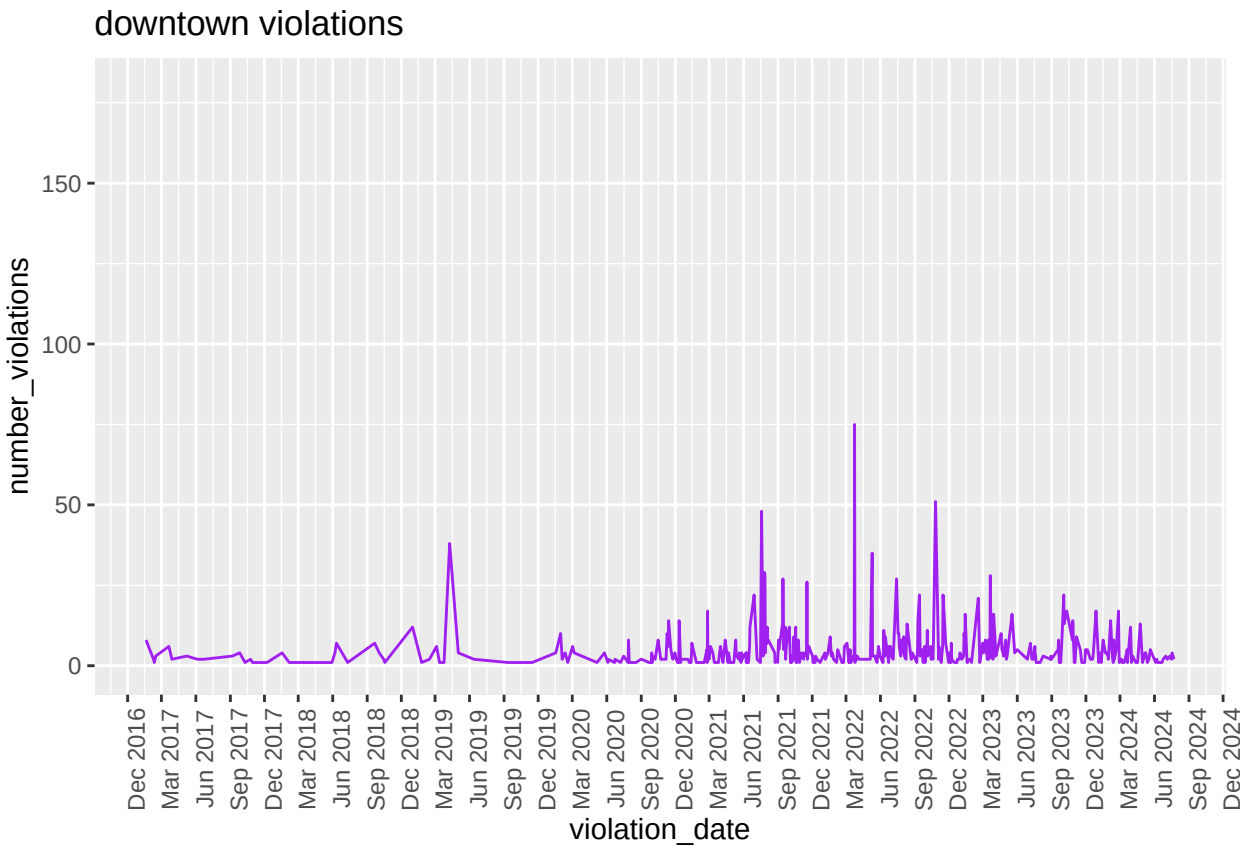


B. Next, generate similar graphs for each of the remaining three subsets. Use a different color for each line plot. For full credit, ensure that the x-axis is displayed in a more granular format, such as quarterly intervals, and set the y-axis to extend up to at least 180, e.g. by using the ylim() function. This will allow for effective comparison of patterns across the different plots.

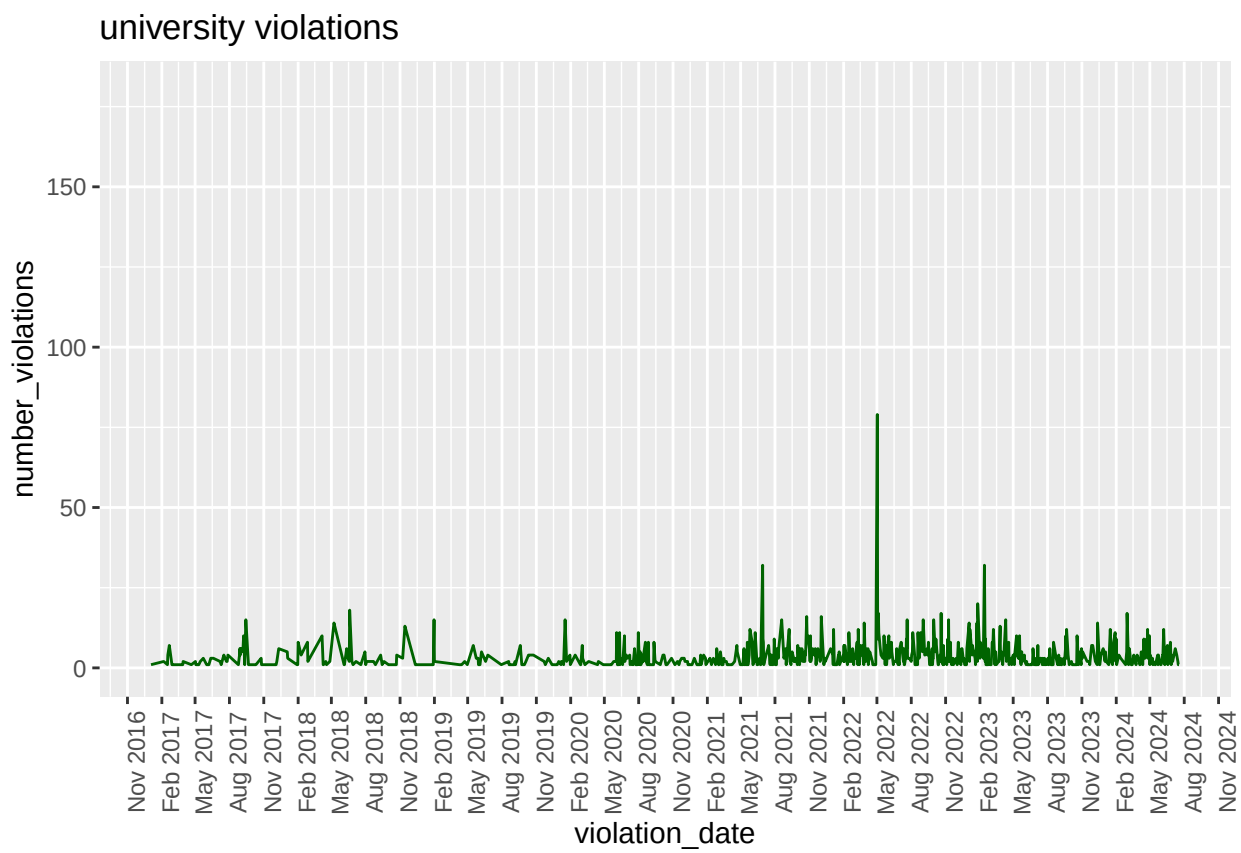
```r
ggplot(date_westcott, aes(x = violation_date, y = number_violations)) + geom_line(color = "lightblue") +
    labs(title = "westcott violations") + ylim(0, 180) + scale_x_date(date_breaks = "3 months",
    date_labels = "%b %Y") + theme(axis.text.x = element_text(angle = 90))
```




```
ggplot(date_downtown, aes(x = violation_date, y = number_violations)) + geom_line(color = "purple") +
    labs(title = "downtown violations") + ylim(0, 180) + scale_x_date(date_breaks = "3 months",
    date_labels = "%b %Y") + theme(axis.text.x = element_text(angle = 90))
```




```
ggplot(date_university, aes(x = violation_date, y = number_violations)) + geom_line(color = "darkgreen")
    labs(title = "university violations") + ylim(0, 180) + scale_x_date(date_breaks = "3 months",
    date_labels = "%b %Y") + theme(axis.text.x = element_text(angle = 90))
```

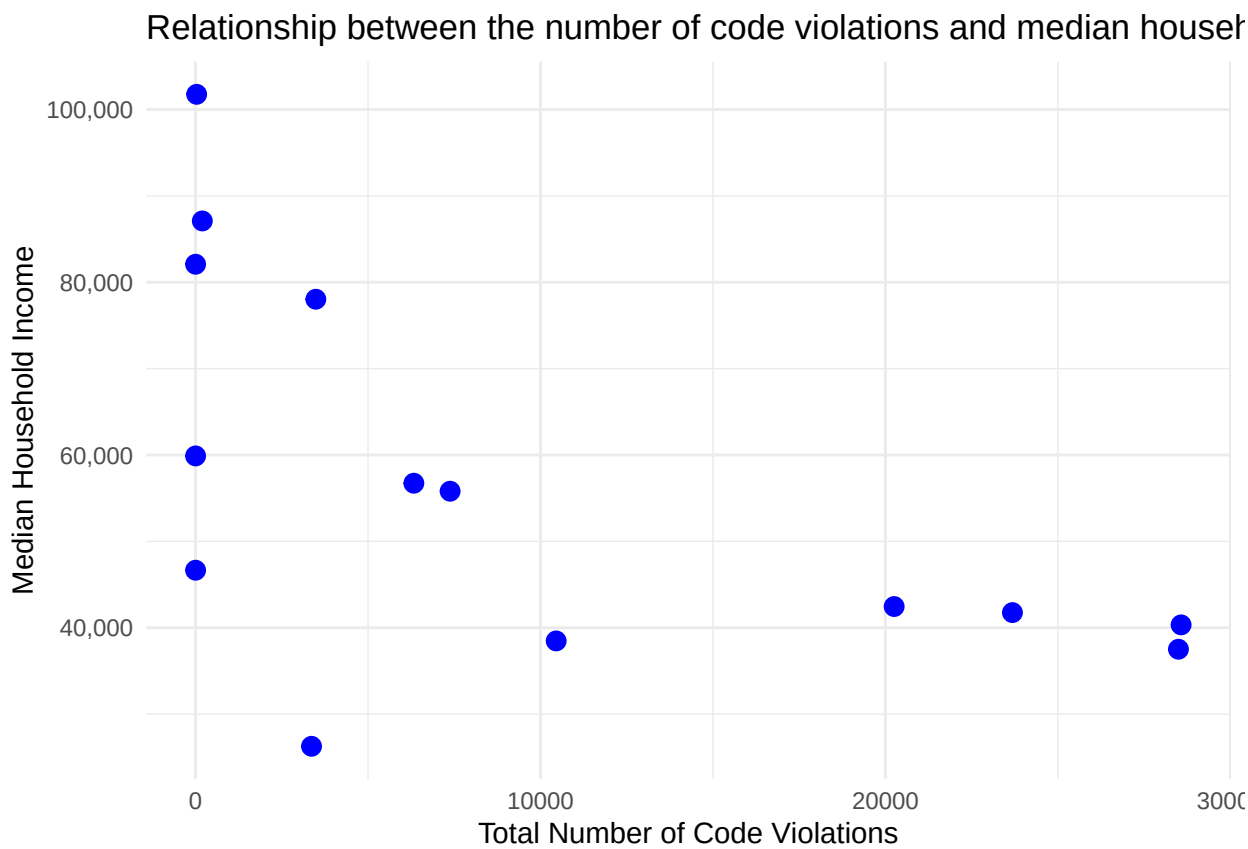


C. Now, let's create a scatter plot using violations_df to explore the relationship between number_violations and zip_median_income. This code calculates the total number of violations for each zip code and visualizes the relationship. Run the code and add comments to explain what each line does.

```
agg_data <- aggregate(violations_df$number_violations, by = list(complaint_zip = violations_df$complain
    zip_median_income = violations_df$zip_median_income), FUN = sum)
# combines the data

colnames(agg_data) <- c("complaint_zip", "median_income", "total_violations")
# makes column names


ggplot(agg_data, aes(x = total_violations, y = median_income)) + geom_point(color = "blue",
    size = 3) + ggtitle("Relationship between the number of code violations and median household income
    xlab("Total Number of Code Violations") + ylab("Median Household Income") + scale_y_continuous(labe
    big.mark = ",", scientific = FALSE)) + theme_minimal()
```




```
# creates a scatterplot to show violations based on median income
```

Step 5: Executive summary

A. In 3-4 sentences, summarize the results from steps 3 and 4 - based on your interpretation of the data, do you see any patterns within and between the line charts and scatterplot? What story do these charts tell?

```
# in the scatterplot, it shows that there are more violations with a lower
# median income. the line charts show that there are the most violations in the
# northside area. This shows that it is likely lower income people living in
# the northside.
```